CampusMatch Architecture

CampusMatch — App Architecture (System Overview)
- 1. Purpose of This Document
- 2. High-Level System Diagram
- 3. Core Architectural Decisions
- 4. User-Facing Flow Map
- 5. Guest Mode — System Behavior
- 6. Cross-Cutting Concerns
  - 6.1 Verification (university-only access)
  - 6.2 Real-time data
  - 6.3 Media pipeline
  - 6.4 Moderation (Block / Report)
  - 6.5 Notifications
- 7. Non-Functional Requirements
- 8. How the Other Two Documents Relate

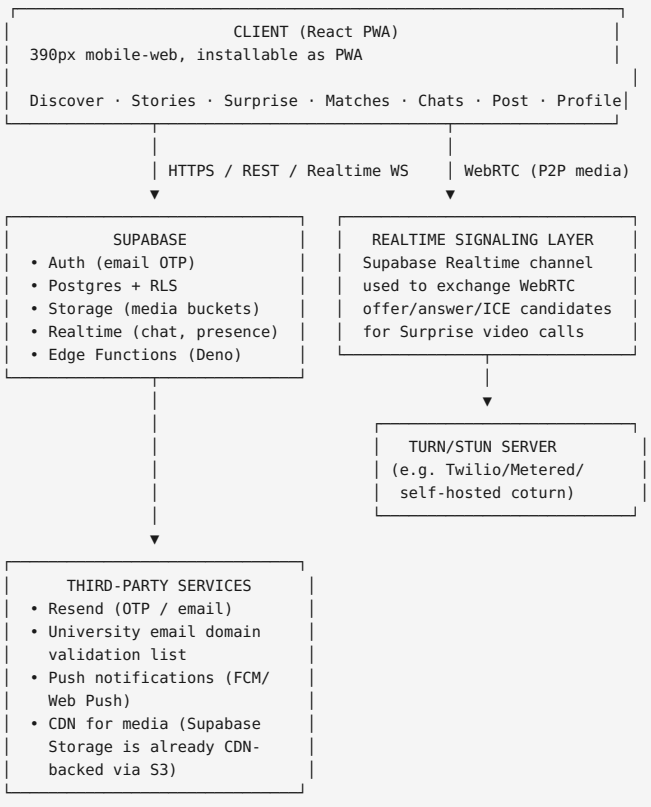
CampusMatch — App Architecture (System Overview)

1. Purpose of This Document

This is the top-level architecture. It describes the whole system end to end — how the mobile-web client, Supabase backend, and third-party services fit together — without going deep into either frontend component structure or backend schema (those live in their own documents). Read this one first.

---

2. High-Level System Diagram



---

3. Core Architectural Decisions

Decision	Choice	Reasoning
Client framework	React + Vite + TypeScript + Tailwind	Matches existing CampusMatch build; fast dev cycle

Backend	Supabase (Postgres, Auth, Storage, Realtime, Edge Functions)	Single vendor, generous free tier, RLS gives row-level security without a custom API layer
Auth	Supabase Auth, email OTP only (no password)	Matches verified-student-only requirement; OTP via Resend tied to university email domains
Video calls (Surprise)	WebRTC peer-to-peer, Supabase Realtime as signaling channel	Avoids needing a dedicated media server for 1:1 calls; cheapest path to launch
Matching queue (Surprise)	Edge Function + Postgres “presence” table	Centralized pairing logic, avoids race conditions of client-side matching
State management (client)	Zustand (or React Query + Zustand)	Lightweight, avoids Redux boilerplate, pairs well with Supabase’s async data model
Image/video editing	Client-side (Canvas API / WebCodecs) before upload	Keeps backend simple — backend only ever stores final processed media
Guest mode	Client-only flag, no Supabase session, gated UI overlays	No backend changes needed; purely a frontend permission gate

---

4. User-Facing Flow Map

```
Landing Page
  │
  ├─ "Get Started" ──► Sign Up (university email + OTP)
  │                        │
  │                        ├─ verified ──► Discover (full access)
  │                        └─ Login link ──► Login (OTP) ──► Discover
  │
  └─ "Continue as Guest" ──► Discover (read-only, interaction-gated)
```

```
Discover (home)
 ├─ Stories bar
 │    ├─ "+" → Post Story screen
 │    └─ tap avatar → View Story screen (own or others)
 ├─ Header icons
 │    ├─ Surprise icon → Surprise Meetup flow
 │    ├─ Notifications icon → Notifications screen
 │    └─ Settings icon → Settings screen
 └─ Feed (vertical scroll of Photo / Video / Gallery cards)
      ├─ tap user header → User Profile (other person)
      ├─ tap 3-dot menu → Block / Report sheet
      └─ like / view profile / caption

Bottom Nav: Discover · Matches · Post (center, raised) · Chats · Profile
```

```
Matches
 ├─ "New matches" (carousel)
 ├─ "Suggested for you" (recommendation engine)
 └─ Matched list → tap → opens Chat thread

Chats
 ├─ Active threads (matched, unlocked)
 ├─ Pending/Frozen threads (≤5 messages sent, no reply yet)
 └─ Chat screen (1:1 messaging)

Post (3-step wizard)
 1. Select  (Photo / Video / Gallery, from library or device)
 2. Edit    (Filters / Adjust / Crop / Text overlay)
 3. Share   (Caption, audience: Everyone / Matches only / My uni)

Profile (My Profile)
 ├─ Edit profile
 ├─ My posts (manage: edit/delete)
 ├─ Liked posts (truncated → "View all")
 ├─ Saved from Surprise (truncated → "View all")
 ├─ Matches preview (truncated → "View all" → Matches screen)
 ├─ Chats preview (truncated → "View all" → Chats screen)
 └─ Settings: Account, Privacy, Blocked accounts, Report history, Logout

User Profile (someone else's)
 ├─ Bio, photos/videos/galleries grid
 ├─ "Send Message" → starts DM (max 5 messages until reply)
 └─ "Like Profile" button
```

---

5. Guest Mode — System Behavior

Guest mode is enforced entirely on the client; there is no Supabase session for guests.

- No `auth.uid()` exists, so any RLS-protected write (like, message, post, story, surprise) is impossible even if someone bypasses the UI.
- Client wraps interactive elements (heart, send message, post, surprise, story post) in a `<RequireAuth>` gate that intercepts the tap and shows "Sign up to continue" instead of forwarding the action.
- Discover feed reads use an anonymous/public Supabase key with RLS policies that allow `SELECT` on public posts only — guests literally cannot fetch matches, chats, or profiles beyond what's public.

---

6. Cross-Cutting Concerns

6.1 Verification (university-only access)

- Sign-up email is checked client-side (fast feedback) and server-side (Edge Function, authoritative) against an allow-list of university email domains stored in a `universities` table.
- OTP is sent via Resend only after domain validation passes.

6.2 Real-time data

Three independent realtime concerns, all riding on Supabase Realtime: 1. **Chat messages** — Postgres changes feed (`messages` table) per thread. 2. **Presence** — who's online/available for Surprise matching. 3. **WebRTC signaling** — ephemeral offer/answer/ICE exchange for Surprise calls, done over a dedicated realtime channel per call session (not stored permanently).

6.3 Media pipeline

```
Device camera/library
  → Client-side edit (filters/crop/adjust/text overlay) via Canvas/WebCodecs
  → Compressed export (image: WebP/JPEG, video: transcoded if needed)
  → Upload to Supabase Storage bucket (posts/, stories/, avatars/)
  → Row inserted in posts/stories table referencing storage path
  → Feed reads via signed/public URL
```

6.4 Moderation (Block / Report)

- Block: client writes to `blocks` table; RLS on every read query excludes blocked pairs in both directions.
- Report: writes to `reports` table; Edge Function or scheduled job aggregates report counts and can auto-flag/ban accounts past a threshold.

6.5 Notifications

- In-app notifications table populated by Postgres triggers (new match, new message, new story view, etc.) and/or Edge Functions.
- Push notifications (Web Push/FCM) dispatched via Edge Function triggered on insert.

---

7. Non-Functional Requirements

Concern	Approach
Security	RLS on every table, no direct service-role key on client, Storage buckets private with signed URLs except public post media
Privacy	University + verified-student gating, block/report enforced at the DB layer, not just UI
Performance	Feed pagination (cursor-based), image lazy-loading, video autoplay only for in-viewport card
Scalability	Stateless Edge Functions, Postgres connection pooling (Supabase pooler), CDN-backed Storage
Offline/PWA	Service worker caches shell + last feed page; guest/auth state persisted locally
Observability	Supabase logs + a lightweight `events` table for key actions (signups, matches, reports) for product analytics

---

8. How the Other Two Documents Relate

- **Frontend Architecture** — folder structure, routing, component breakdown, state management, the post-editing pipeline, and how guest mode is gated in code.
- **Backend Architecture** — full Supabase schema (tables, columns, relationships), RLS policies, Storage bucket layout, Edge Functions, and realtime channel design.